

Rajarata University of Sri Lanka
Department of Computing



ICT1402

PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 10

Arrays

K.A.S.H. Kulathilake

Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

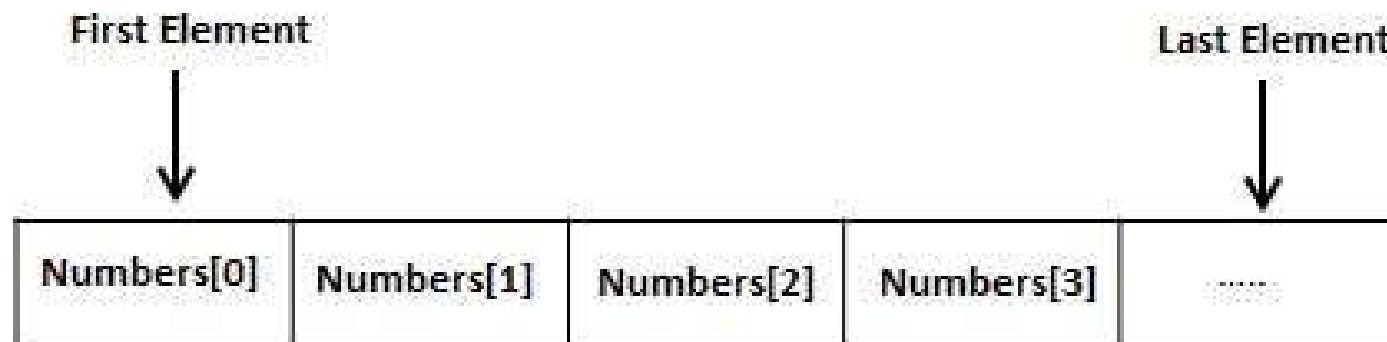
- At the end of this lecture students should be able to;
 - Describe the C arrays.
 - Practice the declaration, initialization and access linear arrays.
 - Practice the declaration, initialization and access two dimensional arrays.
 - Apply taught concepts for writing programs.

Introduction

- The C language provides a capability that enables you to define a set of ordered data items known as an *array*.
- Arrays are a kind of data structure that can store a fixed-size sequential collection of elements of the same type.
- An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.
- Instead of declaring individual variables, such as `number0`, `number1`, ..., and `number99`, you declare one array variable such as `numbers` and use `numbers[0]`, `numbers[1]`, and ..., `numbers[99]` to represent individual variables.
- A specific element in an array is accessed by an index.

Introduction (Cont...)

- All arrays consist of contiguous memory locations.
- The lowest address corresponds to the first element and the highest address to the last element.



One Dimension Array

- A list of items can be given one variable name using only one subscript and such a variable is called a single subscripted variable or a one-dimensional array.
- In C, single-subscripted variable x_i can be expressed as:

$x[1]$, $x[2]$, $x[3]$,, $x[n]$

Declaring Array

- To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array as follows;

type arrayName [arraySize] ;

- This is called a *single-dimensional* array.
- The arraySize must be an integer constant greater than zero and type can be any valid C data type.
- For example, to declare a 5-element array called balance of type double, use this statement;
double balance[5] ;
- Here *balance* is a variable array which is sufficient to hold up to 5 double numbers.

Array Initialization

- Following is an example to assign a single element of the array;
`balance[0] = 1000.0;`
`balance[1] = 2.0;`
`balance[2] = 3.4;`
`balance[3] = 7.0;`
`balance[4] = 50.0;`
- All arrays have 0 as the index of their first element which is also called the base index and the last index of an array will be total size of the array minus 1.

Array Initialization (Cont...)

	0	1	2	3	4
balance	1000.0	2.0	3.4	7.0	50.0

- If you omit the size of the array, an array just big enough to hold the initialization is created.
- Therefore, if you write:

```
double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

- You will create exactly the same array as you did in the previous example.

Array Initialization (Cont...)

- You can declare and initialize an array in C using a single statement as follows:

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

- The number of values between braces { } cannot be larger than the number of elements that we declare for the array between square brackets [].

Accessing Array Elements

- An element is accessed by indexing the array name.
- This is done by placing the index of the element within square brackets after the name of the array.
- For example:
double salary = balance[3];
- The above statement will take the 4th element from the array and assign the value to salary variable.

Demonstration

```
#include <stdio.h>
int main ()
{
    int index;
    double balance[5];
    balance[0] = 1000.0;
    balance[1] = 2.0;
    balance[2] = 3.4;
    balance[3] = 7.0;
    balance[4] = 50.0;

    printf ("Direct Access\n");
    printf ("%6.2f\n ", balance[3]);

    printf ("Loop through Access\n");
    for (index = 0; index < 5; index++)
        printf ("%6.2f\n ", balance[index]);
    return 0;
}
```

Demonstration

```
#include <stdio.h>
int main ()
{
    int index;
    double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};

    for (index = 0; index < 5; index++)
        printf ("%6.2f\n ", balance[index]);
    return 0;
}
```

Demonstration

```
#include <stdio.h>
int main ()
{
    int index;
    double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};

    for (index = 0; index < 5; index++)
        printf ("%6.2f\n ", balance[index]);
    return 0;
}
```

Demonstration

```
#include <stdio.h>
int main ()
{
    int balance[5];
    int index;

    printf ("Before Initialization\n");
    for (index = 0; index < 5; index++)
        printf ("%i\n", balance[index]);

    printf ("\nAfter Initialization\n");
    for (index = 0; index < 5; index++){
        balance[index] = index*10;
        printf ("%i\n ", balance[index]);
    }

    return 0;
}
```

What happen if you
access an array just
after the
declaration:

Replace the following statement for array
declaration in line 4 and see the results?
`int balance[5] = {0,0,0,0,0};`

Demonstration

- Following statements may be used in programs just like any other C variable.

```
aNumber    =  number[0] + 10;  
number [4] = number [0] + number[2] ;  
number [2] = x[5] + y[10] ;  
value[6] = number [i] * 3;
```

- The subscript of an array can be integer constants, integer variables like i, or expressions that yield integers.



Demonstration

```
#include <stdio.h>
int main ()
{
    int index;
    double cost;
    double us_balance[4];
    double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};

    cost = balance[2]+100;
    printf ("Cost ---> %6.2f\n ", cost);

    for (index = 0; index < 4; index++){
        us_balance[index] = balance [index] +
                                   balance [index+1];

        printf ("%6.2f\n ", us_balance[index]);
    }
    return 0;
}
```

Demonstration

```
#include <stdio.h>
int main ()
{
    int index;
    double ot[] = {10.0, 30.0, 35.5, 21.5,20};
    int rate[] = {10,20,30,20,10};
    double gross[5];

    for (index = 0; index < 5; index++){
        gross[index] = ot[index] * rate[index];
        printf ("%6.2f\n", gross[index]);
    }

    return 0;
}
```

Demonstration

```
#include <stdio.h>
int main ()
{
    const int OT_RATE = 10;
    int index;
    double ot[] = {10.0, 30.0, 35.5, 21.5, 20};
    double gross[5];

    for (index = 0; index < 5; index++) {
        gross[index] = ot[index] * OT_RATE;
        printf ("%6.2f\n", gross[index]);
    }

    return 0;
}
```

Bound Checking?

- Declares grades to be an array containing 100 integer elements:
`double grades[100];`
- Valid references to this array can be made by using subscripts from 0 through 99.
- **But be careful to use valid subscripts because C does not do any checking of array bounds for you.**
- So a reference to element number 150 of array grades, as previously declared, does not necessarily cause an error but does most likely cause unwanted, if not unpredictable, program results.

Demonstration

- **Array of Counters**

```
#include <stdio.h>
int main (void)
{
    int ratingCounters[11], i, response;
    for ( i = 1; i <= 10; ++i )
        ratingCounters[i] = 0;
    printf ("Enter your responses\n");
    for ( i = 1; i <= 20; ++i ) {
        scanf ("%i", &response);
        if ( response < 1 || response > 10 )
            printf ("Bad response: %i\n", response);
        else
            ++ratingCounters[response];
    }
    printf ("\n\nRating Number of Responses\n");
    printf ("-----\n");
    for ( i = 1; i <= 10; ++i )
        printf ("%4i%14i\n", i, ratingCounters[i]);
    return 0;
}
```

Demonstration

- **Generating Fibonacci Numbers**

```
#include <stdio.h>
int main (void)
{
    int Fibonacci[15], i;
    Fibonacci[0] = 0;
    Fibonacci[1] = 1;
    for ( i = 2; i < 15; ++i )
        Fibonacci[i] = Fibonacci[i-2] +
                        Fibonacci[i-1];
    for ( i = 0; i < 15; ++i )
        printf ("%i\n", Fibonacci[i]);
    return 0;
}
```

Character Array

```
#include <stdio.h>
int main (void)
{
    char word[] = { 'H', 'e', 'l', 'l', 'o', '!' };
    int i;
    for ( i = 0; i < 6; i++ )
        printf ("%c", word[i]);
    printf ("\n");
    return 0;
}
```



Character Array (Cont...)

- The most notable point in the preceding program is the declaration of the character array word.
- There is no mention of the number of elements in the array.
- The C language allows you to define an array without specifying the number of elements.
- If this is done, the size of the array is determined automatically based on the number of initialization elements.
- Because preceding program has six initial values listed for the array word, the C language implicitly dimensions the array to six elements

Demonstration

- Base Converter (up to 16)

```
#include <stdio.h>
int main (void)
{
    const char baseDigits[16] = {
        '0', '1', '2', '3', '4', '5', '6', '7',
        '8', '9', 'A', 'B', 'C', 'D', 'E', 'F' };
    int convertedNumber[64];
    long int numberToConvert;
    int nextDigit, base, index = 0;
    printf ("Number to be converted? ");
    scanf ("%ld", &numberToConvert);
    printf ("Base? ");
    scanf ("%i", &base);
```

Demonstration

```
do {  
    convertedNumber[index] = numberToConvert % base;  
    ++index;  
    numberToConvert = numberToConvert / base;  
}  
while ( numberToConvert != 0 );  
printf ("Converted number = ");  
for (--index; index >= 0; --index ) {  
    nextDigit = convertedNumber[index];  
    printf ("%c", baseDigits[nextDigit]);  
}  
printf ("\n");  
return 0;  
}
```

Multidimensional Arrays

- The types of arrays that you have been exposed to so far are all linear arrays / single dimension arrays.
- That is, they all dealt with a single dimension.
- The C language allows arrays of any dimension to be defined.
- In this section, you take a look at two-dimensional arrays.

Multidimensional Arrays (Cont...)

- One of the most natural applications for a two-dimensional array arises in the case of a matrix.

10	5	-3	17	82
9	0	0	8	-7
32	20	1	0	14
0	0	8	7	6

- In mathematics, it is quite common to refer to an element of a matrix by use of a double subscript.
- So if you call the preceding matrix M , the notation $M_{i,j}$ refers to the element in the i^{th} row, j^{th} column, where i ranges from 1 to 4, and j ranges from 1 to 5.
- The notation $M_{3,2}$ refers to the value 20, which is found in the 3rd row, 2nd column of the matrix

Multidimensional Arrays (Cont...)

- In C, you can use an analogous notation when referring to elements of a two dimensional array.
- However, because C likes to start numbering things at zero, the 1st row of the matrix is actually row 0, and the 1st column of the matrix is column 0.
- The preceding matrix would then have row and column designations, as shown below;

Column (j)	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>Row (i)</u>					
0	10	5	-3	17	82
1	9	0	0	8	-7
2	32	20	1	0	14
3	0	0	8	7	6

Multidimensional Arrays (Cont...)

- Whereas in mathematics the notation $M_{i,j}$ is used, in C the equivalent notation is `M[i][j]`.
- Remember, the first index number refers to the row number, whereas the second index number references the column.
- So the statement

```
int sum = M[0][2] + M[2][4];
```

 - adds the value contained in row 0, column 2—which is -3 —to the value contained in row 2, column 4—which is 14 —and assigns the result of 11 to the variable `sum`.
 - Two-dimensional arrays are declared the same way that one-dimensional arrays are; thus;

```
int M[4][5];
```

 - declares the array `M` to be a two-dimensional array consisting of 4 rows and 5 columns, for a total of 20 elements.
 - Each position in the array is defined to contain an integer value.

Multidimensional Arrays (Cont...)

- Two-dimensional arrays can be initialized in a manner analogous to their one-dimensional counterparts.
- When listing elements for initialization, the values are listed by row.
- Brace pairs are used to separate the list of initializers for one row from the next.
- So to define and initialize the array M to the elements listed in preceding table, a statement such as the following can be used:

```
int M[4][5] = {  
    { 10, 5, -3, 17, 82 },  
    { 9, 0, 0, 8, -7 },  
    { 32, 20, 1, 0, 14 },  
    { 0, 0, 8, 7, 6 }  
};
```

Multidimensional Arrays (Cont...)

```
#include <stdio.h>
int main (void)
{
    int M[4][5] = {
        { 10, 5, -3, 17, 82 },
        { 9, 0, 0, 8, -7 },
        { 32, 20, 1, 0, 14 },
        { 0, 0, 8, 7, 6 }
    };

    int x, y;

    for ( x = 0; x < 4; x++){
        for (y = 0; y < 5; y++){
            printf( "%2i. ", M[x][y]);
        }
        printf ("\n");
    }
    return 0;
}
```



Multidimensional Arrays (Cont...)

- As with one-dimensional arrays, it is not required that the entire array be initialized.

- A statement such as

```
int M[4][5] = {  
    { 10, 5, -3 },  
    { 9, 0, 0 },  
    { 32, 20, 1 },  
    { 0, 0, 8 }  
};
```

- Only initializes the first three elements of each row of the matrix to the indicated values.
- The remaining values are set to 0.

Multidimensional Arrays (Cont...)

```
#include <stdio.h>
int main (void)
{
    int M[4][5] = {
        { 10, 5, -3 },
        { 9, 0, 0 },
        { 32, 20, 1 },
        { 0, 0, 8 }
    };

    int x, y;

    for ( x = 0; x < 4; x++){
        for (y = 0; y < 5; y++)
            printf( "%2i. ", M[x][y]);
        printf ("\n");
    }
    return 0;
}
```



Multidimensional Arrays (Cont...)

- Note that commas are required after each brace that closes off a row, except in the case of the final row.
- The use of the inner pairs of braces is actually optional.
- If not supplied, initialization proceeds by row.
- Thus, the preceding statement could also have been written as follows:

```
int M[4][5] = { 10, 5, -3, 17, 82, 9, 0, 0, 8, -7,  
32, 20, 1, 0, 14, 0, 0, 8, 7, 6 };
```

Multidimensional Arrays (Cont...)

```
#include <stdio.h>
int main (void)
{
    int M[4][5] = { 10, 5, -3, 17, 82, 9, 0,
                    0, 8, -7, 32, 20, 1, 0, 14, 0, 0, 8, 7, 6 };
    int x, y;

    for ( x = 0; x < 4; x++) {
        for (y = 0; y < 5; y++)
            printf( "%2i. ", M[x][y]);
        printf ("\n");
    }
    return 0;
}
```

Variable Length Arrays

- In the examples we discussed up to now, you have seen how the size of an array is declared to be of a specific size.
- The C language allows you to declare arrays of a variable size.
- Note that, full support for variable-length arrays was not offered by all C compilers.
- You might want to check your compiler's documentation before you use this feature.

Objective Re-cap

- Now you should be able to:
 - Describe the C arrays.
 - Practice the declaration, initialization and access linear arrays.
 - Practice the declaration, initialization and access two dimensional arrays.
 - Apply taught concepts for writing programs.

References

- Chapter 07, - Programming in C, 3rd Edition, Stephen G. Kochan

Q & A

Next: Working with Functions